

Frequency Count Method :

Frequency Count Method assigns 1 unit of time to each statement and counts execution frequency to determine algorithm time complexity.

Key Principle :

Time taken = Σ (frequency of each statement * 1 unit).

Order of n :

Highest degree term in the resulting polynomial determines complexity (e.g., order of n , n^2 , n^3).

Array Sum Algorithm (Order of n) :

Algorithm :

Sum all elements in array of size n

Execution Frequencies :

Initialization : 1 time
Loop Condition ($i < n$) : $n + 1$ times
Loop Body ($\text{sum} += a[i]$) : n times
Increment ($i++$) : n times
Final Statement : 1 time

Time Function :

$f(n) = 2n + 2 \approx \text{order of } n \text{ (degree 1)}.$

Space Complexity :

Order of n (array size n + few scalars).

Matrix Sum Algorithm : (Order of n^2)

Algorithm :

Sum of two $n * n$ matrices

Nested Loop Analysis :

1. **Outer loop (i) :**
n+1 Condition checks, n body executions.
2. **Inner loop (j) :**
n+1 condition checks, n body executions.
3. **Body statements :**
2 statements * n * n = $2n^2$

Time Function :

$f(n) = 2n^2 + 2n + 2 \rightarrow$ order of n^2 (degree 2).

Space Complexity :

$3n^2$ (A, B, C matrices) + 2 scalars $\rightarrow$ order of n^2 .

Matrix Multiplication Algorithm : (Order of n^3)

Algorithm :

Multiply two $n \times n$ matrices.

Triple Nested Loop Analysis :

Outer i loop : n+1 checks, n executions
middle j loop : n+1 checks, n executions
inner k loop : n+1 checks, n executions

Critical Statement `(c[i][j] += a[i][k] * b[k][j])` : Executes n^3 times.

Time Function: $f(n) = 3n^3 + 3n^2 + 2n + 1$ $f(n) = 3n^3 + 3n^2 + 2n + 1 \rightarrow$ order of n^3 (degree 3).

Space Complexity: $3n^2$ (A, B, C) + 3 scalars $\rightarrow$ order of n^2 .

Key Pattern Summary :

Algorithm	Loops	Time	Space	Highest Degree
Array sum	1	n	n	1
Matrix sum	2	n^2	n^2	2
Matrix mul	3	n^3	n^2	3

Rule :

k nested loops → order of n^k time complexity.